LLM-Native Applications: Toward a New Paradigm in Software Engineering

Alan Portela Bandeira
GESAD
State University of Ceará - UECE
Fortaleza, Ceará, Brazil
alan.portela@uece.br

Gabriel Luiz Barros De Oliveira
GESAD
State University of Ceará - UECE
Fortaleza, Ceará, Brazil
gabrielluiz.barros@aluno.uece.br

Paulo Henrique Mendes Maia
GESAD
State University of Ceará - UECE
Fortaleza, Brazil
pauloh.maia@uece.br

Carlos Aderaldo
University of Fortaleza
Fortaleza, Ceará, Brazil
cmendesce@gmail.com

Nabor Mendonca
University of Fortaleza
Fortaleza, Ceará, Brazil
nabor@unifor.br

Abstract

Large Language Models (LLMs) are increasingly embedded into software systems, yet most current solutions are either *LLM-based*, wrapping an existing system around an LLM, or *LLM-augmented*, embedding LLMs into existing workflows. These approaches yield quick gains but remain brittle, difficult to reproduce, and poorly governed. We introduce the notion of *LLM-native applications*: systems conceived from the outset with an LLM as the orchestration core, coordinating tools, memory, knowledge integration, and governance. We present a reference architecture and illustrate feasibility through an LLM-native copilot for literature reviews. We further propose a lifecycle model that highlights how development processes change when LLMs become first-class orchestrators.

CCS Concepts

• **Software and its engineering** → **Software architectures**; **Layered systems**; **Abstraction, modeling and modularity**; • **Computing methodologies** → *Natural language generation*; *Probabilistic reasoning*.

Keywords

LLM-Native, Large Language Models, LLM, Software Architecture

ACM Reference Format:
Alan Portela Bandeira, Gabriel Luiz Barros De Oliveira, Paulo Henrique Mendes Maia, Carlos Aderaldo, and Nabor Mendonca. 2026. LLM-Native Applications: Toward a New Paradigm in Software Engineering. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE-Companion '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/3774748.3795680

1 Introduction

Context. Large Language Models (LLMs) are transforming how software systems are conceived, built, and used [4]. Their ability to generate, explain, and orchestrate text-based interactions has led to rapid adoption. Nonetheless, software engineering is still adapting which role these models should play in engineered applications [2].

Motivation. LLM models are usually integrated in software through systems that serves and exposes API's. This type of model management has favoured two main approaches: *LLM-based applications*, which exposes the model as the main engine, such as chatbots or summarizers, and *LLM-augmented applications*, which embed an LLM as a peripheral service that enhances existing workflows, as in IDEs or retrieval-augmented assistants [8]. While impactful, both approaches may introduce Architectural Technical Debt (ATD) [3] due to incremental design [7]. Architectural reflections increasingly call for principled approaches to design LLM-enabled systems that go beyond local integration [1].

Problem. Therefore, an architectural model that favours the design from the ground up with the LLM as the organizing principle is missing. Without clear definition, reference architecture, and development lifecycle, LLMs are underutilized as "add-ons" for classic architectures. This gap is particularly pressing given the rising complexity of tasks delegated to LLMs and the governance, reproducibility, and adaptability challenges they entail [1].

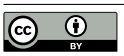
Our Approach. We introduce the concept of *LLM-native applications*, which represent a paradigm shift: they are conceived from day one around the model's reasoning, memory, governance, knowledge integration, and orchestration capabilities, requiring fundamentally different architectural thinking, since they are treated now as first-class concerns.

2 The Use of LLMs in Software Development

We distinguish three modes LLM-enabled systems, each reflecting a different architectural stance.

LLM-based applications. These systems directly expose the capabilities of an LLM, often via APIs, without substantial additional logic. ChatGPT and Gemini are good exemplars. While highly accessible, their architecture tends to be shallow, often limited to simple prompt–response loops.

LLM-augmented applications. These are applications that incorporate LLMs as a peripheral service to enhance existing functionality. At design time, the model supplements the development of the software itself but does not orchestrate its architecture. Examples include GitHub Copilot suggesting code within IDEs or assistants



This work is licensed under a Creative Commons Attribution 4.0 International License.
ICSE-Companion '26, Rio de Janeiro, Brazil
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2296-7/26/04
https://doi.org/10.1145/3774748.3795680

integrated into productivity suites. Augmentation benefits utility but preserves legacy structures, the LLM acts in a subordinate role.

LLM-native applications. Distinct from the incrementalism of the first two categories, LLM-native applications are conceived from the outset with the LLM at the centre of the architecture. The LLM acts as the orchestrator: it interprets user intent, generates and executes plans, invokes external tools, integrates knowledge, manages memory, and enforces governance. Other components exist to serve this orchestration, not merely to wrap or extend it.

3 Reference Architecture

A general representation of the architecture can be seen in Figure 1. The main layers and components are described as follows.

User Interface (UI). Users interact via text, voice, or multimodal inputs, managing intermediate plans, retrieved evidence, and governance warnings.

LLM Core. The central organizer interprets intent, decomposes tasks, reasons over alternatives, and orchestrates component interactions. The Core produces a *plan* that structures tool calls and knowledge access; plans are human-inspectable and versioned for replay. Unlike augmentation, the Core governs reasoning and execution flow, consolidating accountability in a single locus of control.

Governance & Safety Layer. Governance validates plans against policies such as privacy constraints, requirements, and bias checks.

Application Orchestration. Orchestrator invokes tools, schedules calls, manages retries, and results. Failures trigger bounded retries and re-planning by the Core, favouring resilience.

Knowledge Base (KB). Retrieval pipelines (e.g., RAG [5]) fetch relevant documents from structured and unstructured sources, supplying evidence for reasoning.

Tool & Service Layer. Specialized tasks (e.g., PDF parsing, code execution, data cleaning) are delegated to external services via typed interfaces. The Core invokes them and integrates structured results. Function calling and self-instructed tool use ensure schema validation, bounded retries, and sandboxed execution [6].

Context & Memory Manager. Manages context and persistent, versioned memory to ensure continuity and reproducibility.

The execution goes as: (1) the user submits a request via UI; (2) the Core interprets it and proposes a plan; (3) governance validates the plan against policies; (4) orchestration executes tool and KB calls; (5) Core reconciles results into outputs; (6) governance performs post-execution validation and (7) the UI delivers the output, and the relevant context is persisted in memory.

LLM-native lifecycles adopt an *LLM-first* stance from day one: the LLM Core orchestrates plans, tools, memory, retrieval, and governance, which forces process changes across requirements, design, implementation, testing, deployment, and evolution. Table 1 highlights the differences between types of LLM-enabled applications.

To illustrate feasibility, we developed **SCRIBE** [1], an LLM-native copilot for systematic literature reviews. Researchers submit queries such as *"Summarize recent approaches to LLMs in software testing"*. The *LLM Core* interprets the request, generates a plan, and orchestrates retrieval, filtering, and synthesis under governance checks before producing a structured report with citations.

[1]https://github.com/gesad-science/SCRIBE-CREW

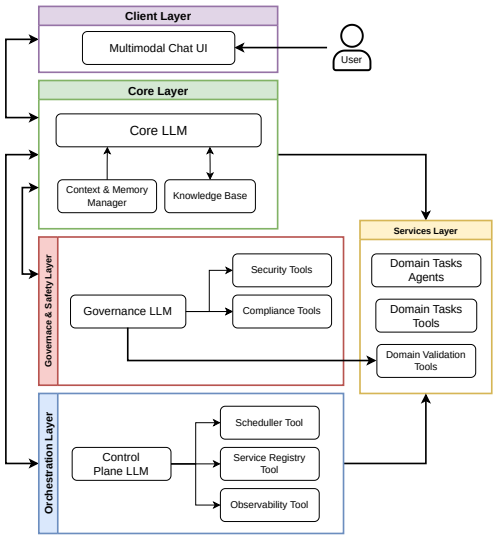


Figure 1: LLM-native layered architecture view

Dimension	Traditional	LLM-Based	LLM-Augmented	LLM-Native
Requirements	Functional specs	Functional specs + prompt/use cases	Legacy specs + added LLM features	Specs include Core orchestration, governance, transparency
Design	Components + APIs	Minimal design around LLM API	Legacy design + LLM sidecar (e.g., RAG)	Core-centric layered design (tools, memory, retrieval, governance)
Implementation	Code + DB schemas	API wrappers, prompt templates	Glue code + retrieval/filter/validators	MCP-wrapped tools, A2A agents, plans, validators as code
Testing	Unit, integration, acceptance	Output checks, prompt regression	+ Retrieval and safety filter tests	Hook-based: usability, reasoning, reproducibility, retrieval, governance
Deployment	APIs + DBs	API keys, rate limits, logs	+ Retrieval infra & filter monitors	+ Event sourcing/provenance, vector DBs, validator agents, plan/memory services
Evolution	Patch code	Prompt tweaks; provider model updates	Prompts + retrievers/indices + filters	Continuous adaptation of models, plans, policies, memories (with version pinning)
Roles	Developers, testers	+ Prompt crafters	+ Retrieval engineers, safety maintainers	+ Prompt/UX designers, governance officers, data curators, eval engineers

Table 1: Lifecycle comparison across traditional, LLM-based, LLM-augmented, and LLM-native applications.

References

[1] Alessio Bucaioni, Martin Weyssow, Junda He, Yunbo Lyu, and David Lo. 2025. A Functional Software Reference Architecture for LLM-Integrated Systems. In *22nd IEEE International Conference on Software Architecture Companion*. 1–5.

[2] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M Zhang. 2023. Large language models for software engineering: Survey and open problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE, 31–53.

[3] Lucas Graham, Dale Dzielski, and Yasser Alshehri. 2024. Architecture Technical Debt Management and Prioritization: A Systematic Mapping Study. In *International Conf. on Computational Science and Computational Intelligence*. Springer, 263–281.

[4] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79.

[5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances*

in neural information processing systems 33 (2020), 9459–9474.

[6] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems* 36 (2023), 68539–68551.

[7] Zhiyuan Wan, Yun Zhang, Xin Xia, Yi Jiang, and David Lo. 2023. Software architecture in practice: Challenges and opportunities. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1457–1469.

[8] Yutao Zhu, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhan Liu, Chenlong Deng, Haonan Chen, Zheng Liu, Zhicheng Dou, and Ji-Rong Wen. 2025. Large Language Models for Information Retrieval: A Survey. *ACM Trans. Inf. Syst.* (Sept. 2025). doi:10.1145/3748304 Just Accepted.